Assignment 3 Priority Email GenAI Analysis

1. GenAI Used
   The two Generative AIs that I used for this assignment are Perplexity AI accessed through its website (perplexity.ai) through the base model and ChatGPT accessed through its respective website (chatgpt.com).
2. Code Analysis
   Correctness: Both programs implement a class-based MaxHeap and correctly prioritize emails by sender category and newest date. ChatGPT uses procedural code reading from a file while Perplexity uses standard manipulation methods. Both use argv[1] to take inputs. Both handle insertion, removal, and viewing the highest-priority email.

   Execution Time: Both programs have O(log n) insertion and removal, and O(1) NEXT and COUNT operations. Both use an efficient MaxHeap. This is the same execution time as the C based code.

   Space Complexity: Both use O(n) space for storing emails and O(1) auxiliary space for heap operations. Neither uses recursive heap functions. ChatGPT limits its capacity of max emails at 1000 while Perplexity uses a dynamically resizing array and doubles capacity when needed.

   Maintainability: ChatGPT has more detailed comments and uses inline comments, multiline comments, and provided prologue comments. It also organizes code through global variables.. Perplexity has no comments and does a better job of encapsulating the code through object oriented programming. Both codes offer room for improvement on their maintainability.

3. GenAI Used
   I selected the Perplexity program as the basis for my assignment. It more efficiently uses classes for better memory management and heap operations. It correctly prioritizes emails by category and date. Its separate heap functions make the code easy to understand and modify. It also allows me to write the comments as opposed to the ChatGPT code that already has comments.
4. Improvements
   Correctness: Created an executive class to better encapsulate the program. Additionally created more classes to serve their functions. Added getSender(), getSubject(), and getDate() for a more proper encapsulation technique.

   Execution Time: Olog(n) Execution time which is the same as the raw source code and the C code from the previous assignment. Calculating the priority number and date integer right when the email is created ensures that heap comparisons stay extremely fast at $O(1)$ constant time.

   Space Complexity: Memory is kept at $O(n)$ by only doubling the dynamic array when it gets full, and auxiliary space stays at $O(1)$ because the heapify functions use while loops instead of recursion to save stack space.

   Maintainability: Added prologue comments and added inline comments to each line to better the readability of the code. Inline comments are added to each line to help analyze each line and

explain to the reader what the code is doing.

5. Raw Code
   This is the code that immediately came out of my prompt. The same prompt was used for both GenAI which is just the standalone [Assignment 3 Instructions.pdf] upload followed by a prompt "lets get started" for both Programs.
   Raw ChatGPT Code

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_EMAILS 1000
#define MAX_STR 256

typedef struct {
    char sender[64];
    char subject[256];
    char date[16];
    int priority;
    int dateKey;
} Email;

Email heap[MAX_EMAILS];
int heapSize = 0;

int getCategoryPriority(char* category) {
    if (strcmp(category, "Boss") == 0) return 5;
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;
    return 0;
}

int parseDateKey(char* date) {
    int month, day, year;
    sscanf(date, "%d-%d-%d", &month, &day, &year);
    return year * 10000 + month * 100 + day;
}
```

```
int compare(Email a, Email b) {
    if (a.priority != b.priority) return a.priority - b.priority;
    return a.dateKey - b.dateKey;
}

void swap(Email* a, Email* b) {
    Email temp = *a;
    *a = *b;
    *b = temp;
}

void heapifyUp(int index) {
    while (index > 0) {
        int parent = (index - 1) / 2;
        if (compare(heap[index], heap[parent]) > 0) {
            swap(&heap[index], &heap[parent]);
            index = parent;
        } else {
            break;
        }
    }
}

void heapifyDown(int index) {
    while (1) {
        int left = 2 * index + 1;
        int right = 2 * index + 2;
        int largest = index;
        if (left < heapSize && compare(heap[left], heap[largest]) >
0)
            largest = left;
        if (right < heapSize && compare(heap[right], heap[largest])
> 0)
            largest = right;
        if (largest != index) {
            swap(&heap[index], &heap[largest]);
            index = largest;
        } else {
            break;
        }
```

```
    }
}

void insertEmail(Email e) {
    if (heapSize >= MAX_EMAILS) return;
    heap[heapSize] = e;
    heapifyUp(heapSize);
    heapSize++;
}

Email* peekEmail() {
    if (heapSize == 0) return NULL;
    return &heap[0];
}

Email removeEmail() {
    Email empty = {"", "", "", 0, 0};
    if (heapSize == 0) return empty;
    Email top = heap[0];
    heap[0] = heap[heapSize - 1];
    heapSize--;
    if (heapSize > 0) heapifyDown(0);
    return top;
}

void trim(char* str) {
    int len = strlen(str);
    while (len > 0 && (str[len-1] == '\n' || str[len-1] == '\r' ||
str[len-1] == ' ')) {
        str[--len] = '\0';
    }
    int start = 0;
    while (str[start] == ' ') start++;
    if (start > 0) {
        memmove(str, str + start, len - start + 1);
    }
}

int main(int argc, char* argv[]) {
    FILE* file = fopen(argv[1], "r");
```

```
    if (!file) {
        printf("Could not open file.\n");
        return 1;
    }

    char line[512];
    while (fgets(line, sizeof(line), file)) {
        trim(line);
        if (strlen(line) == 0) continue;

        if (strncmp(line, "EMAIL", 5) == 0) {
            char* rest = line + 6;
            char* comma1 = strchr(rest, ',');
            if (!comma1) continue;
            *comma1 = '\0';
            char* category = rest;
            char* rest2 = comma1 + 1;
            char* comma2 = strrchr(rest2, ',');
            if (!comma2) continue;
            *comma2 = '\0';
            char* subject = rest2;
            char* date = comma2 + 1;

            Email e;
            strcpy(e.sender, category);
            strcpy(e.subject, subject);
            strcpy(e.date, date);
            e.priority = getCategoryPriority(category);
            e.dateKey = parseDateKey(date);
            insertEmail(e);
        } else if (strcmp(line, "NEXT") == 0) {
            Email* e = peekEmail();
            if (e) {
                printf("Next email:\n");
                printf("Sender: %s\n", e->sender);
                printf("Subject: %s\n", e->subject);
                printf("Date: %s\n", e->date);
            } else {
                printf("No emails to read.\n");
            }
```

```
        } else if (strcmp(line, "READ") == 0) {
            removeEmail();
        } else if (strcmp(line, "COUNT") == 0) {
            printf("There are %d emails to read.\n", heapSize);
        }
    }

    fclose(file);
    return 0;
}
```

Raw Perplexity Code

```
#include <iostream>
#include <fstream>
#include <string>
#include <cstring>

using namespace std;

class Email {
public:
    string sender;
    string subject;
    string date;
    int priority;
    int dateKey;

    Email() : sender(""), subject(""), date(""), priority(0),
dateKey(0) {}

    int getPriority(string category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson") return 1;
        return 0;
    }
```

```
    int parseDate(string d) {
        int month, day, year;
        sscanf(d.c_str(), "%d-%d-%d", &month, &day, &year);
        return year * 10000 + month * 100 + day;
    }

    void setInfo(string cat, string sub, string dt) {
        sender = cat;
        subject = sub;
        date = dt;
        priority = getPriority(cat);
        dateKey = parseDate(dt);
    }
};

class MaxHeap {
private:
    Email* data;
    int size;
    int capacity;

    void resize() {
        capacity *= 2;
        Email* newData = new Email[capacity];
        for (int i = 0; i < size; i++) {
            newData[i] = data[i];
        }
        delete[] data;
        data = newData;
    }

    bool hasHigherPriority(Email& a, Email& b) {
        if (a.priority != b.priority) return a.priority >
b.priority;
        return a.dateKey > b.dateKey;
    }

    void swap(Email& a, Email& b) {
        Email temp = a;
        a = b;
```

```
        b = temp;
    }

    void heapifyUp(int index) {
        while (index > 0) {
            int parent = (index - 1) / 2;
            if (hasHigherPriority(data[index], data[parent])) {
                swap(data[index], data[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        while (true) {
            int left = 2 * index + 1;
            int right = 2 * index + 2;
            int largest = index;
            if (left < size && hasHigherPriority(data[left],
data[largest]))
                largest = left;
            if (right < size && hasHigherPriority(data[right],
data[largest]))
                largest = right;
            if (largest != index) {
                swap(data[index], data[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }

public:
    MaxHeap() {
        capacity = 16;
        size = 0;
        data = new Email[capacity];
```

```
    }

    ~MaxHeap() {
        delete[] data;
    }

    void insert(Email e) {
        if (size >= capacity) resize();
        data[size] = e;
        heapifyUp(size);
        size++;
    }

    Email* peek() {
        if (size == 0) return nullptr;
        return &data[0];
    }

    void removeTop() {
        if (size == 0) return;
        data[0] = data[size - 1];
        size--;
        if (size > 0) heapifyDown(0);
    }

    int count() {
        return size;
    }
};

int main(int argc, char* argv[]) {
    if (argc < 2) {
        cout << "Usage: " << argv[0] << " <input_file>" << endl;
        return 1;
    }

    ifstream file(argv[1]);
    if (!file.is_open()) {
        cout << "Could not open file." << endl;
        return 1;
```

```
    }

    MaxHeap heap;
    string line;

    while (getline(file, line)) {
        // Remove trailing whitespace
        while (!line.empty() && (line.back() == '\r' || line.back()
== ' ')) {
            line.pop_back();
        }
        if (line.empty()) continue;

        if (line.substr(0, 6) == "EMAIL ") {
            string rest = line.substr(6);
            int comma1 = rest.find(',');
            if (comma1 == string::npos) continue;
            string category = rest.substr(0, comma1);
            int comma2 = rest.find_last_of(',');
            if (comma2 == string::npos || comma2 == comma1)
continue;
            string subject = rest.substr(comma1 + 1, comma2 - comma1
- 1);
            string date = rest.substr(comma2 + 1);

            Email e;
            e.setInfo(category, subject, date);
            heap.insert(e);
        } else if (line == "NEXT") {
            Email* e = heap.peek();
            if (e) {
                cout << "Next email:" << endl;
                cout << "Sender: " << e->sender << endl;
                cout << "Subject: " << e->subject << endl;
                cout << "Date: " << e->date << endl;
            } else {
                cout << "No emails to read." << endl;
            }
        } else if (line == "READ") {
            heap.removeTop();
```

```
        } else if (line == "COUNT") {
            cout << "There are " << heap.count() << " emails to
read." << endl;
        }
    }

    file.close();
    return 0;
}
```